Dr. Konrad Völkel Heinrich-Heine-Universität Düsseldorf
Wintersemester 2026/27

Einführung in Deep Learning
Blatt 1
(11 Punkte, abzugeben bis 12:00 am Mittwoch, dem 21.10.2026)

Jede Aufgabe gibt einen Punkt, auch Aufgabe 4b – insgesamt 11 Punkte. Sie müssen also mindestens 6 Aufgaben mehr oder weniger korrekt lösen (die Hälfte der Punkte), um dieses Übungsblatt zu bestehen.

Geben Sie eine PDF-Datei und Python-Code (wahlweise im Notebook) ab.

Hinweise. *Das Blatt wiederholt lineare Algebra: Skalarprodukt, Orthogonalität, Länge und Winkel, und setzt das in NumPy und Matplotlib um. Erinnern Sie sich an die Interpretation des Kosinus eines Winkels als Verhältnis von Seiten im rechtwinkligen Dreieck: Mit Ankathete a und Hypotenuse b ist $\cos(\alpha) = a/b$. Im Spezialfall $b = 1$ ist $\cos(\alpha) = a$, d.h. der Kosinus ist die Länge der Projektion eines Einheitsvektors auf die Richtung des anderen Schenkels. Genau das steckt in der Formel*

$$\cos(\alpha) = \frac{u \cdot v}{\|u\| \, \|v\|}$$

für den Winkel zwischen zwei Vektoren $u, v \in \mathbb{R}^n$: Das Skalarprodukt $w \cdot x$ im Perzeptron misst, "wie sehr x in Richtung w zeigt" (Kapitel "Neuron und Perzeptron" im Skript). Broadcasting in NumPy:
`https://voelkel.pages.cs.uni-duesseldorf.de/ds-skript/numpy#broadcasting`

Lineare Algebra: Winkel

Satz. *Für zwei Vektoren $v, w \in \mathbb{R}^d$ mit Längen 1 ist $v \cdot w \in [-1, 1]$. Dabei ist $v \cdot w = 1$ genau dann wenn $v = w$, entsprechend $v \cdot w = -1$ genau dann wenn $v = -w$.*

Es gilt sogar: $v \cdot w = \cos(\angle(v, w))$.

Beweisskizze (rechnen Sie nach, machen Sie sich den Sachverhalt klar). Es genügt, den Beweis für $d = 2$ zu führen, da ansonsten ein Koordinatenwechsel zur von v und w aufgespannten Ebene uns in diese Situation versetzt. Eine Rotation bringt uns in die Situation $w = e_1$. Wir zeichnen v als Punkt im Einheitskreis und fällen das Lot auf die x-Achse. Der auf der x-Achse getroffene Punkt bildet mit dem Ursprung und v ein rechtwinkliges Dreieck. Die x-Koordinate ist genau $v \cdot e_1$, aber außerdem auch $\cos(\angle(v, e_1))$.

Nun müssen wir uns noch davon überzeugen, dass die Rotation und zuvor die Projektion auf die vw-Ebene nichts am Ergebnis ändert.

Wenn v und w in die gleiche Richtung zeigen, ist der Winkel 0, somit der Kosinus des Winkels 1. Wenn v und w außerdem gleich lang sind, sind die Vektoren gleich. Das Vorzeichen in $v \cdot -v$ lässt sich linear herausziehen, also $v \cdot -v = -(v \cdot v) = -\|v\|^2 = -1^2 = -1$.

Der Kosinus einer reellen Zahl liegt stets zwischen -1 und $+1$ einschließlich. □

Aufgabe 1. *Schreiben Sie den oben skizzierten Beweis sauber auf und füllen Sie die Lücken in der Argumentation (Begründungen für die unbegründeten Aussagen). Alternativ können Sie einen eigenen Beweis des Satzes aufschreiben.*

Aufgabe 2. *Zeichnen Sie die Werte der Kosinusfunktion von -2π bis 2π (von Hand). Tragen Sie für ganzzahlige Vielfache von $\frac{\pi}{2}$ präzise Werte ein. Der einzige Wert außer $0, \pm 1$, den man sich merken kann (und der das Zeichnen erleichtert), ist $\cos\left(\frac{\pi}{4}\right) = \frac{\sqrt{2}}{2} \approx 0.7$.*

Einführung in Deep Learning
Blatt 1
(11 Punkte, abzugeben bis 12:00 am Mittwoch, dem 21.10.2026)

Beispiel. *Bei einem* 90*-Winkel, wir schreiben auch anstelle von (*90 *von* 360*) gern (*$\frac{\pi}{2}$ *von* 2π*), gilt also für* $\angle(v,w) = \frac{\pi}{2}$ *dass* $v \cdot w = \cos\left(\frac{\pi}{2}\right) = 0$ *ist. Damit ist* orthogonal *also nur ein fancy Wort für einen rechten Winkel. Ein Winkel zwischen* 0 *und* 90 *entspricht den Werten* $\cos(0) = 1$ *bis* $\cos\left(\frac{\pi}{2}\right) = 0$*, ein Winkel zwischen* 90 *und* 180 *entspricht den Werten* $\cos\left(\frac{\pi}{2}\right) = 0$ *bis* $\cos(\pi) = -1$.

Aufgabe 3. *Zeichnen Sie zwei Vektoren* $v = \begin{pmatrix}4\\3\end{pmatrix}$ *und* $w = \begin{pmatrix}3\\4\end{pmatrix}$ *in der Ebene ein (von Hand). Berechnen Sie die Längen* $\|v\|, \|w\|$ *der Vektoren. Zeichnen Sie einen Einheitskreis und kennzeichnen Sie die normierten Vektoren* $\widetilde{v}, \widetilde{w}$*. Zeichnen Sie ein rechtwinkliges Dreieck mit einer Seite* v *und einer Seitenlänge* $\cos(\angle(v,w))$*.*

Aufgabe 4. *Begründen Sie:* $(v \cdot \widetilde{w})\widetilde{w}$ *ist die Projektion von* v *auf die von* w *aufgespannte Gerade. Diese Projektion erhält man auch, indem man von* v *das Lot auf diese Gerade fällt (also* v *so mit der* w*-Gerade verbindet, dass die Verbindungslinie senkrecht auf der* w*-Geraden steht).*

Aufgabe 4b. *Sei* $w = \begin{pmatrix}3\\4\end{pmatrix}$ *und* $b = -5$*. Zeichnen Sie die Gerade* $\{w \cdot x + b = 0\}$ *und den Vektor* w *(von Hand). Berechnen Sie für die Punkte* $(3,4)$*,* $(-1,0)$ *und* $(1,\frac{1}{2})$ *jeweils* $w \cdot x + b$ *und den Abstand zur Geraden, und tragen Sie die Punkte ein. Was ändert sich an Gerade, Aktivierungen (*$\zeta := w \cdot x + b$*) und Abständen (*$\zeta/\|w\|$*), wenn Sie* w *und* b *beide mit* 10 *multiplizieren?*

Einführung in Deep Learning
Blatt 1
(11 Punkte, abzugeben bis 12:00 am Mittwoch, dem 21.10.2026)

Implementierung in Numpy

Verwenden Sie keine vorgefertigten Methoden aus Numpy, lediglich die Datenstruktur `ndarray`. Sie müssen hier kein Type Checking, Error Handling oder Broadcasting implementieren.

Aufgabe 5. *Implementieren Sie eine Funktion `hadamard(v,w)`, die zwei Numpy Arrays v, w von gleichem Shape annimmt und einen Numpy Array vom gleichen Shape zurückgibt, der dem Hadamard-Produkt von $v \odot w$ entspricht. Vereinfachte Variante: Nur Shape $(n,)$ erlaubt.*

Schreiben Sie mit dem `assert`-Schlüsselwort einen 'Testcase', der mit fest gewählten Vektoren aus der Ebene prüft, ob die Methode korrekt arbeitet.

Aufgabe 6. *Implementieren Sie eine Funktion `scalprod(v,w)`, die zwei Numpy Arrays v, w von gleichem Shape $(n, m_1, \ldots, m_k)$ annimmt und einen Numpy Array von Shape $(m_1, \ldots, m_k)$ zurückgibt, der 'entlang der ersten Achse' das Skalarprodukt ausrechnet, also $\sum_{i=1}^{n}(v \odot w)_i$.*

Vereinfachte Variante: $k = 0$, also nur Shape $(n,)$ erlaubt, sodass der Rückgabewert ein Skalar (Shape $()$) ist.

Schreiben Sie auch hier einen Testcase wie bei der vorherigen Aufgabe, aber prüfen Sie hier nicht nur auf Gleichheit mit dem zuvor bekannten Ergebnis, sondern auch auf Gleichheit mit dem Ergebnis von `np.dot(v,w)`.

Aufgabe 7. *Implementieren Sie eine Funktion `isOrthogonal(v,w)`, die zwei Numpy Arrays v, w von gleichem Shape annimmt und einen Boolean zurückgibt, der dem Wert von $[v \cdot w = 0]$ entspricht. Nutzen Sie Ihre eigene Implementierung von `scalprod`.*

Aufgabe 8. *Implementieren Sie eine Funktion `getOrthogonal(v)`, die einen Numpy Array v nimmt und einen Numpy Array w zurückgibt, der orthogonal auf v steht.*

Testen Sie Ihre Implementierung mit Hilfe der zuvor implementierten Methode `isOrthogonal`.

Aufgabe 9. *Implementieren Sie eine Funktion `getNormalRandomPlanar()`, die einen Numpy Array von Shape $(2,)$ zurückgibt (also $(x, y)^\top \in \mathbb{R}^2$), dessen Komponenten x, y aus einer eindimensionalen Standardnormalverteilung gesampelt sind.*

Dazu dürfen Sie `numpy.random.normal` benutzen.

Visualisierung mit Matplotlib

Aufgabe 10. *Generieren Sie mit der zuvor implementierten `getNormalRandomPlanar()` einen zufälligen Vektor $v = (x, y)^\top \in \mathbb{R}^2$. Zeichnen Sie diesen Vektor als Pfeil vom Ursprung zum Punkt (x, y) mit `matplotlib.pyplot` ein.*

Zeichnen Sie in den selben Plot auch den (ggf. kürzeren oder längeren) Pfeil zu $\widetilde{v} = \frac{v}{\|v\|}$ ein.

Zeichnen Sie in den selben Plot die Ursprungsgerade ein, die senkrecht auf $\widetilde{v}$ bzw. v steht.

Generieren Sie einen weiteren zufälligen Vektor $w \in \mathbb{R}^2$ und entscheiden Sie, ob $v \cdot w > 0$ oder $v \cdot w < 0$ ist. Verwenden Sie zur Kennzeichnung von w im Plot einen Kreis, falls > 0 und ein Kreuz, falls < 0. Zeichnen Sie auf diese Weise 9 weitere zufällige Punkte.

Erklären Sie, was Sie beobachten können. Erklären Sie, was der Begriff "Entscheidungsgrenze" in diesem Zusammenhang bedeuten kann.